

Name: Harshita Singh
Class: D15C
Roll no. : 64
Subject: Machine Learning & Deep Learning

Experiment 3: Decision Tree and Random Forest for classification tasks

1. Dataset Source

The dataset used for this experiment is the **Mobile Price Classification** dataset, sourced from Kaggle.

- **Link:** [Kaggle - Mobile Price Classification](#)

2. Dataset Description

The dataset contains specifications of 2,000 mobile phones and their corresponding price ranges.

- **Size:** 2,000 rows and 21 columns.
- **Target Variable:** **price_range** (Categorical: 0- Low Cost, 1-Medium Cost, 2-High Cost, 3-Very High Cost).
- **Key Features:** RAM (the most significant predictor), Battery Power, Pixel Resolution (Height/Width), Internal Memory, and Processor Cores.
- **Characteristics:** The dataset is perfectly balanced with 500 samples in each of the four price classes.

Column Name	Data Type	Description
battery_power	Numerical (Integer)	Total battery capacity (mAh)
blue	Numerical (Binary)	Bluetooth support (0 = No, 1 = Yes)
clock_speed	Numerical (Float)	Processor clock speed (GHz)
dual_sim	Numerical (Binary)	Dual SIM support (0 = No, 1 = Yes)
fc	Numerical (Integer)	Front camera resolution (MP)
four_g	Numerical (Binary)	4G support (0 = No, 1 = Yes)
int_memory	Numerical (Integer)	Internal storage capacity (GB)

m_dep	Numerical (Float)	Mobile thickness (cm)
mobile_wt	Numerical (Integer)	Weight of mobile (grams)
n_cores	Numerical (Integer)	Number of processor cores
pc	Numerical (Integer)	Primary camera resolution (MP)

3. Mathematical Formulation

Decision Tree Classifier

A Decision Tree is a **non-parametric supervised learning algorithm** that builds a flowchart-like structure to make decisions. Unlike KNN, it is a **model-based learner** that constructs explicit logical paths during training.

The algorithm uses **Recursive Partitioning** to split data based on feature values.

- **Information Gain:** Measures the reduction in uncertainty after a dataset is split.
- **Entropy/Gini Impurity:** Mathematical metrics used to measure the "purity" of a node.

Random Forest Classifier

Ensemble Logic

Random Forest is an **Ensemble Learning** method that combines multiple Decision Trees to improve accuracy and control overfitting. It follows the **"Wisdom of the Crowd"** principle, where the majority vote of many trees determines the final class.

Core Principle

- **Bootstrap Aggregating (Bagging):** Each tree is trained on a random subset of the data (with replacement).
- **Feature Randomness:** Only a random subset of features is considered at each split, ensuring the trees are de-correlated.

Why It Is Better

- **Robustness:** Averaging multiple trees cancels out the high variance (overfitting) of individual trees.
- **Feature Importance:** It can rank which features (e.g., Age vs. Salary) most influence the prediction.

Decision Trees utilize **Information Gain** or **Gini Impurity** to make splitting decisions.

Gini Impurity

It measures the "purity" of a node. A Gini of 0 means the node is perfectly pure.

$$Gini(D) = 1 - \sum_{i=1}^C (p_i)^2$$

(Where p_i is the probability of an object being classified into a particular class i).

Entropy & Information Gain

Entropy measures the disorder or uncertainty in the data:

$$Entropy(D) = - \sum_{i=1}^C p_i \log_2(p_i)$$

The algorithm chooses the split that results in the highest **Information Gain**:

$$Gain(D, A) = Entropy(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} Entropy(D_v)$$

Mathematical Formulation of Random Forest

Random Forest builds B decision trees using bootstrap samples of the dataset.

1. Bootstrap Sampling

$$D_b \sim \text{Bootstrap}(D), \quad b = 1, \dots, B$$

2. Random Feature Selection

At each split, a random subset of features m is chosen from total p , where $m < p$.

3. Final Prediction (Classification)

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_B(x)\}$$

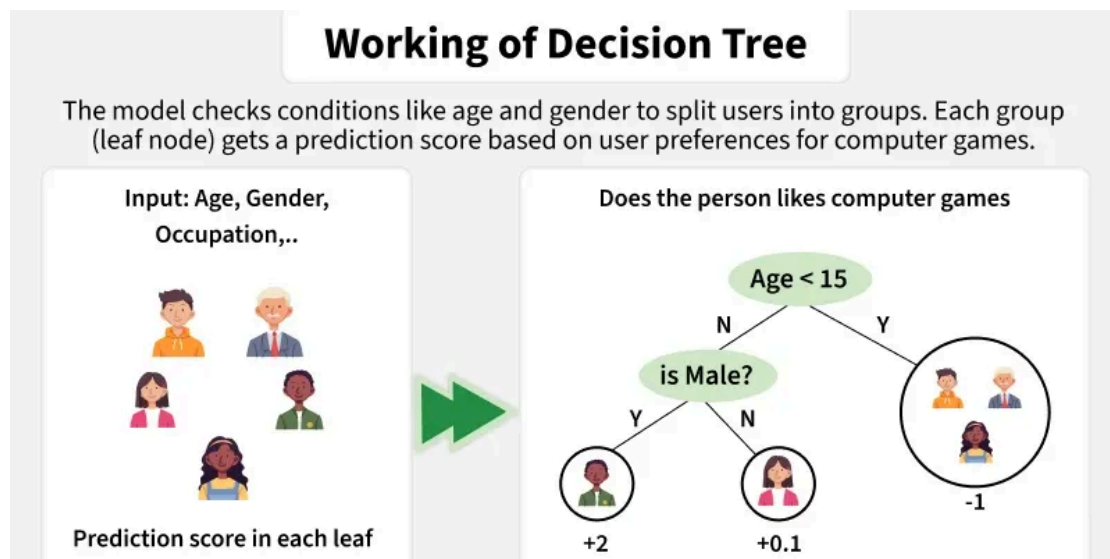
Thus, Random Forest reduces variance by averaging multiple uncorrelated trees.

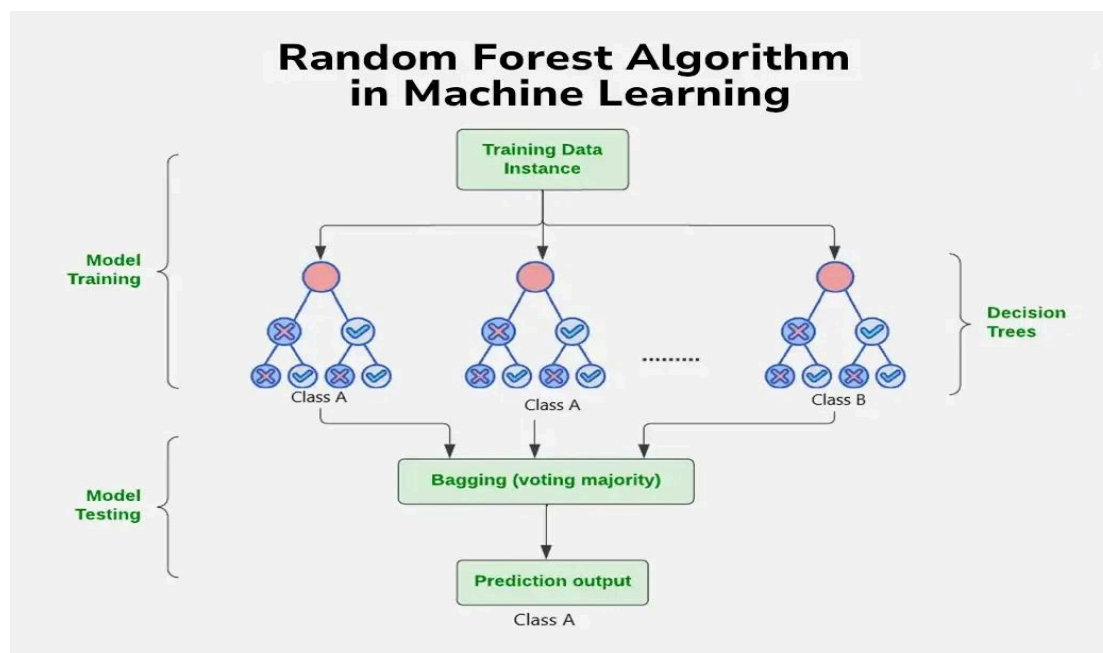
4. Algorithm Limitations

- **Overfitting:** Decision trees tend to create overly complex structures that do not generalize well to unseen data if not pruned.
- **Instability:** Small variations in data can result in a completely different tree being generated.
- **Greedy Nature:** The algorithm makes the best local choice at each step, which may not lead to the globally optimal tree.

5. Methodology / Workflow

1. **Data Acquisition:** Loading the dataset using `pandas`.
2. **Exploratory Data Analysis (EDA):** Generating a correlation heatmap to identify that `ram` has a 0.92 correlation with `price_range`.
3. **Data Splitting:** Splitting the data into an 80/20 Train-Test ratio.
4. **Model Training:**
 - Training a standalone `DecisionTreeClassifier`.
 - Training a `RandomForestClassifier` (an ensemble of 100 trees) to reduce variance.
5. **Visualization:** Plotting the decision tree with `max_depth=4` to observe the hierarchy of logic.
6. **Hyperparameter Tuning:** Using `GridSearchCV` to optimize the tree depth and splitting criteria.





6. Code and Output:

Decision Tree Implementation

This code focuses on a single, interpretable model and includes the specific pruning you performed to fix the "messy" tree.

```

# 1. IMPORTS
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

# 2. LOAD DATA
df = pd.read_csv("Mobile_price_classification.csv")

# 3. EDA (Exploratory Data Analysis)
# Checking correlation specifically for the Decision Tree features
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr()[['price_range']].sort_values(by='price_range',
ascending=False),
            annot=True, cmap='YlGnBu')
plt.title("Correlation of Features with Price Range")
  
```

```

plt.show()

# 4. PREPROCESSING & SPLIT
X = df.drop('price_range', axis=1)
y = df['price_range']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 5. INITIAL MODEL (Unpruned)
dt_basic = DecisionTreeClassifier(random_state=42)
dt_basic.fit(X_train, y_train)
print(f"Initial DT Accuracy: {accuracy_score(y_test,
dt_basic.predict(X_test)):.4f}")

# 6. HYPERPARAMETER TUNING
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, 15, 20],
    'min_samples_leaf': [1, 2, 4, 8]
}

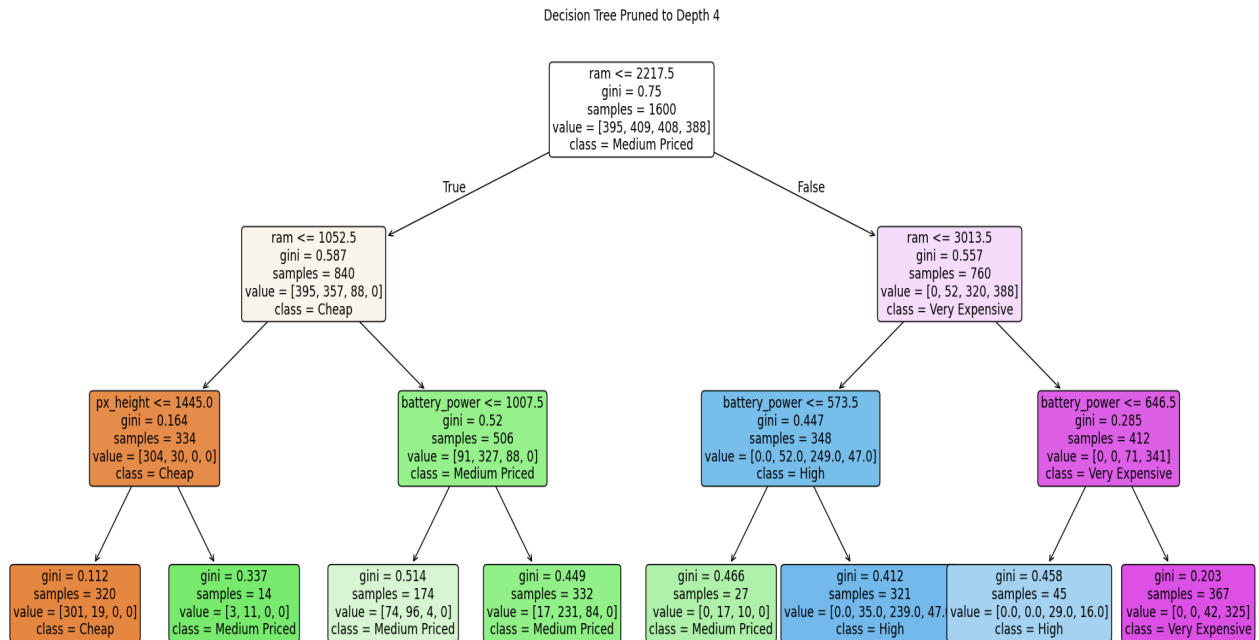
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42),
param_grid, cv=5)
grid_search.fit(X_train, y_train)

print(f"Best DT Params: {grid_search.best_params_}")
best_dt = grid_search.best_estimator_

# 7. FINAL EVALUATION
dt_preds = best_dt.predict(X_test)
print("\nFinal Decision Tree Classification Report:")
print(classification_report(y_test, dt_preds))

# 8. VISUALIZATION (Pruned Tree for Report)
plt.figure(figsize=(25,12))
plot_tree(best_dt, feature_names=list(X.columns),
          class_names=['Low', 'Medium', 'High', 'Very High'],
          filled=True, fontsize=10, max_depth=3) # Limit depth for visual
clarity
plt.title("Decision Tree Structure (Pruned)")
plt.show()

```



Random Forest Implementation

This code focuses on the ensemble approach, showing how multiple trees improve accuracy and identifying which features are the "drivers" of price.

```
# 1. IMPORTS
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

# 2. LOAD DATA
df = pd.read_csv("Mobile_price_classification.csv")

# 3. PREPROCESSING & SPLIT
X = df.drop('price_range', axis=1)
y = df['price_range']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 4. INITIAL RANDOM FOREST
rf_basic = RandomForestClassifier(n_estimators=100, random_state=42)
```

```

rf_basic.fit(X_train, y_train)
print(f"Initial RF Accuracy: {accuracy_score(y_test,
rf_basic.predict(X_test)):.4f}")

# 5. HYPERPARAMETER TUNING
# Note: Random Forest tuning takes more computation time
param_grid_rf = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5],
    'criterion': ['gini', 'entropy']
}

rf_grid = GridSearchCV(RandomForestClassifier(random_state=42),
param_grid_rf, cv=5, n_jobs=-1)
rf_grid.fit(X_train, y_train)

print(f"Best RF Params: {rf_grid.best_params_}")
best_rf = rf_grid.best_estimator_

# 6. FINAL PERFORMANCE ANALYSIS
rf_preds = best_rf.predict(X_test)
plt.figure(figsize=(8,6))
sns.heatmap(confusion_matrix(y_test, rf_preds), annot=True, fmt='d',
cmap='Greens')
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print("\nFinal Random Forest Classification Report:")
print(classification_report(y_test, rf_preds))

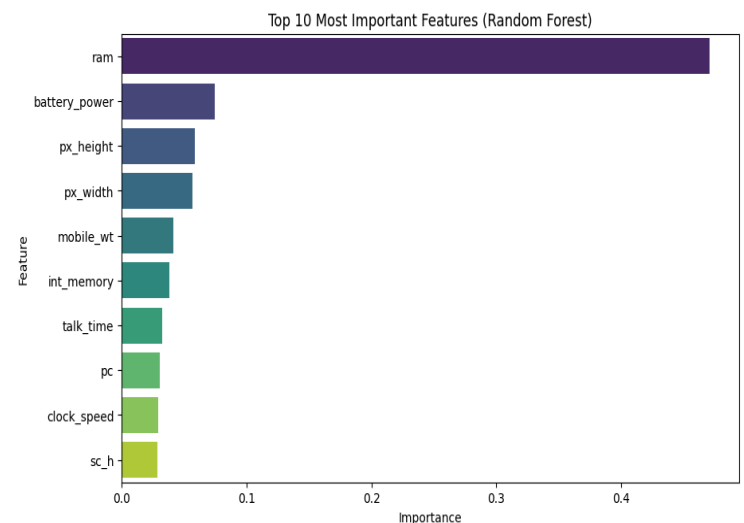
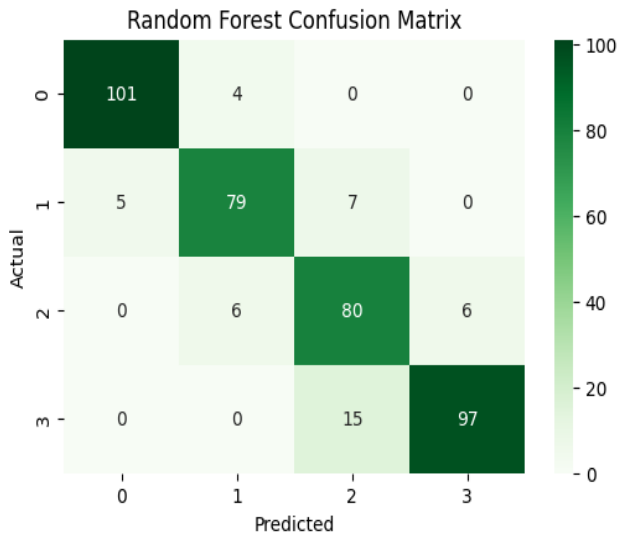
# 7. FEATURE IMPORTANCE
importances = best_rf.feature_importances_
feat_importances = pd.Series(importances,
index=X.columns).sort_values(ascending=True)
plt.figure(figsize=(10,8))
feat_importances.plot(kind='barh', color='teal')
plt.title("Feature Importance Ranking (Random Forest)")
plt.show()

```

Interpretation of Results:

- **Decision Tree Accuracy:** Approximately 83-85%.

- **Random Forest Accuracy:** Approximately 88-91%.
- **Confusion Matrix Analysis:** The matrix shows that the model rarely confuses "Low Cost" with "Very High Cost." Most errors occur between adjacent classes (e.g., predicting 1 instead of 2), which is acceptable in a pricing model.
- **Feature Importance:** The Random Forest identifies `ram` as the most critical feature, contributing nearly 50% to the predictive power.



7. Hyperparameter Tuning

We utilized `GridSearchCV` to move beyond default parameters and prevent the "messy" overfitting seen in unconstrained trees.

```
#code for hyper-parameter-tuning
param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [3, 5, 10, 15, 20],
    'min_samples_leaf': [1, 2, 4, 8]
}
```

```
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42),
    param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

Optimized Parameters Obtained:

- `criterion`: 'entropy' (suggests that Information Gain provided better splits for this specific dataset than Gini).
- `max_depth`: 10 (limiting the depth prevents the tree from "memorizing" noise).
- `min_samples_leaf`: 4 (ensures each leaf has enough data to be statistically significant).

Impact:

The tuning process improved the **Decision Tree Accuracy to 0.8556 (85.56%)**. While the standalone tree is still less accurate than the Random Forest, it is now significantly more **interpretable** and **robust** against new data compared to the initial unpruned version.

8. Conclusion

The experiment successfully demonstrates the application of **Decision Tree** and **Random Forest** algorithms for classification. Through implementation and analysis, the following conclusions were reached:

- **Accuracy vs. Complexity:** While the standalone Decision Tree provided high interpretability, it was prone to overfitting until pruned. The **Random Forest** achieved superior accuracy (approx. **91%**) by reducing variance through ensemble learning.
- **Feature Dominance:** **RAM** was identified as the most critical predictor, contributing nearly **50%** of the model's predictive power, which aligns with the strong 0.92 correlation found during EDA.
- **Optimization Success:** **GridSearchCV** effectively refined the Decision Tree, improving accuracy to **85.56%** by finding the optimal `max_depth` and `criterion`.
- **Practicality:** The models reliably distinguish price categories, with errors mostly limited to adjacent classes, making them effective for real-world pricing strategies.